```
require 'rails_helper'

RSpec.describe User do
  describe "validations" do
    it "requires first_name to be set" do
      expect(subject.valid?).to_not be
      expect(subject.errors[:first_name].size).to eq(1)
    end

    it "requires last_name to be set" do
      expect(subject.valid?).to_not be
      expect(subject.errors[:last_name].size).to eq(1)
    end
  end
end
```

Now when we run the tests again:

```
$ docker-compose exec web bin/rails spec
```

we see our specs correctly failing because we haven't implemented any validations on the User model:

```
Failures:

 1) User validations requires first_name to be set
    Failure/Error: expect(subject.valid?).to_not be
      expected true to evaluate to false
    # ./spec/models/user_spec.rb:6:in `block (3 levels) in <top (required)>'

 2) User validations requires last_name to be set
    Failure/Error: expect(subject.valid?).to_not be
      expected true to evaluate to false
    # ./spec/models/user_spec.rb:11:in `block (3 levels) in <top (required)>'

Finished in 0.09403 seconds (files took 17.39 seconds to load)
2 examples, 2 failures
```

Let's make these tests pass by updating our User model (app/models/user.rb) to look like this:

```
class User < ApplicationRecord
  validates_presence_of :first_name, :last_name
end
```

Now when we rerun the specs:

```
$ docker-compose exec web bin/rails spec
```

we can see that they pass: